Week 6: Gradients, MATLAB, and Research Reading

From Multi-Input Functions to Numerical Jacobians, Matrix Benchmarks, and Paper Understanding

Instructor / Mentor

Kiki Research Training Program

Week 6

Contents

Finish Week 5: Gradient Intuition

Today's Class: Three Missions

1. **Finish the gradient concept:** what changes when a function has many inputs but one output?
2. **Learn basic MATLAB coding:** arrays, matrices, function handles, finite differences, and visualization.
3. **Start deep paper reading:** MIT thesis plus three student-selected papers using Problem / Method / Result.

Big connection

A gradient is not just a calculus object. It is the language behind optimization, machine learning training, and many engineering simulations.

Multi-Input, Single-Output Functions

One output score from many input knobs

A scalar function can take a vector input:

$$f : \mathbb{R}^n \to \mathbb{R}, \qquad y = f(\mathbf{x}).$$

The input is a point:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

- Many input directions exist.
- The function still returns one number.
- Question: at this point, which direction increases output fastest?

Smart-grid example

$$f(T, p, b) = \text{overload risk}$$

- T: temperature
- p: electricity price
- b: battery state

Output: one risk score.

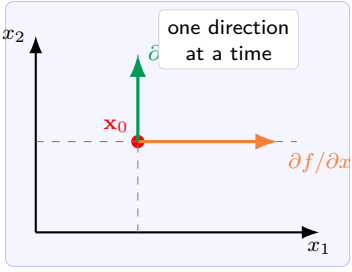
Partial Derivatives: Change One Knob at a Time

At point $\mathbf{x}_0$

The partial derivative with respect to x_i asks:

$$\frac{\partial f}{\partial x_i}(\mathbf{x}_0) = \text{slope if only } x_i \text{ moves.}$$

- Freeze all other inputs.
- Move a tiny step in one direction.
- Measure how the output changes.



Gradient: All Partial Derivatives in One Vector

For $f : \mathbb{R}^n \to \mathbb{R}$

The gradient at an input point $\mathbf{x}_0$ is

$$\nabla f(\mathbf{x}_0) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}_0) \\ \frac{\partial f}{\partial x_2}(\mathbf{x}_0) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}_0) \end{bmatrix}.$$

- Direction: fastest increase.
- Length: how fast it increases.
- Negative direction: fastest decrease.

AI connection

If $L(\theta)$ is model loss, then training often uses

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla L(\theta_{\text{old}}).$$

Concrete Example: Demand Risk Function

Suppose

$$f(T,p) = T^2 + 3Tp + p^2.$$

Think of T as temperature stress and p as price response.

$$\frac{\partial f}{\partial T} = 2T + 3p,$$
$$\frac{\partial f}{\partial p} = 3T + 2p.$$

Therefore,

$$\nabla f(T,p) = \begin{bmatrix} 2T + 3p \\ 3T + 2p \end{bmatrix}.$$

At $(T,p) = (1,2)$

$$\nabla f(1,2) = \begin{bmatrix} 8 \\ 7 \end{bmatrix}.$$

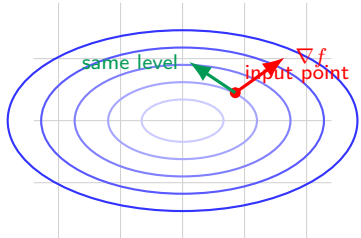
Interpretation: near this point, the output is slightly more sensitive to temperature than price.

Contour Map: Why the Gradient Has a Direction

Contour lines

A contour line connects points with the same output value.

- Walking along a contour means no change in f.
- The gradient points most directly uphill.
- Therefore, the gradient crosses contour lines instead of following them.



gradient is local: it belongs to one point

Gradient vs. Jacobian: Do Not Mix Them Up

Object	Function type	What it stores
Gradient	$f : \mathbb{R}^n \to \mathbb{R}$	One partial derivative per input; usually written as a column vector.
Jacobian	$F : \mathbb{R}^n \to \mathbb{R}^m$	One row per output and one column per input.
Scalar-output Jacobian	$f : \mathbb{R}^n \to \mathbb{R}$	A $1 \times n$ row of partial derivatives; it contains the same information as the gradient.

Simple memory trick

Gradient: one output. Jacobian: possibly many outputs.

MATLAB Basics for Matrix Thinking

MATLAB Mindset: Everything Is an Array

Why MATLAB today?

MATLAB makes matrix and vector operations feel like the default language.

- Great for quick numerical experiments.
- Strong built-in matrix syntax.
- Easy plotting for engineering intuition.

```
clear; clc; close all;

a = 5;       % semicolon hides output
b = 2;
c = a + b  % no semicolon prints
    result

whos         % show variables
help sin     % command-line help
doc sin      % documentation window
```

Vectors, Matrices, and Indexing

```
v = [1 2 3 4];       % row vector
w = [1; 2; 3; 4];    % column vector

A = [1 2 3;
     4 5 6;
     7 8 9];
```

```
A(1,1)       % first row, first column
A(2,3)       % second row, third column
A(:,1)       % all rows, first column
A(2,:)       % second row, all columns
A(1:2,2:3)   % rows 1-2, cols 2-3
```

Important MATLAB habit

MATLAB indexing starts at 1, not 0.

Matrix Operators vs. Element-Wise Operators

Matrix operations

```
A * B       % matrix multiplication
A^2         % matrix power
A \ b       % solve A*x = b
```

Element-wise operations

```
A .* B      % multiply entry by entry
A.^2        % square every entry
A ./ B      % divide entry by entry
```

Class checkpoint

Why does $x.^2$ work for a whole vector, but x^2 may fail when x is not a square matrix?

Function Handles: The Pattern for Gradients

For derivative code, use vector input

Write functions as $f(\mathbf{x})$, where $\mathbf{x}$ is a column vector.

```
% Scalar-output function: R^2 -> R
f = @(x) x(1)^2 + 3*x(1)*x(2) + x(2)^2;

point = [1; 2];
value = f(point);

disp(value)
```

Why this style matters

If every function accepts a column vector, one numerical-gradient function can work for 2 inputs, 20 inputs, or 2,000 inputs.

Numerical Gradients and Jacobians

Finite Differences: Derivatives Without Symbolic Algebra

Central-difference formula

For a tiny step h and unit direction e_i,

$$\frac{\partial f}{\partial x_i}(\mathbf{x}_0) \approx \frac{f(\mathbf{x}_0 + he_i) - f(\mathbf{x}_0 - he_i)}{2h}.$$

- Move forward a tiny amount.
- Move backward a tiny amount.
- Compare the output difference.

Practical note

$h = 10^{-6}$ is a useful starting point, but the best step size depends on the function and computer precision.

MATLAB Lab 1: Numerical Gradient at One Point

```
clear; clc;

% f: R^2 -> R
f = @(x) x(1)^2 + 3*x(1)*x(2) + x(2)^2;

x0 = [1; 2];       % input point
h = 1e-6;          % finite-difference step
n = length(x0);    % number of inputs
grad = zeros(n,1);

for i = 1:n
    e = zeros(n,1);
    e(i) = 1;
    grad(i) = (f(x0 + h*e) - f(x0 - h*e)) / (2*h);
end

disp('Numerical gradient:')
disp(grad)   % expected: approximately [8; 7]
```

Turn the Gradient Code into a Reusable Function

Create a file named `numericalGradient.m`

```
function grad = numericalGradient(f, x0, h)
    n = length(x0);
    grad = zeros(n,1);

    for i = 1:n
        e = zeros(n,1);
        e(i) = 1;
        grad(i) = (f(x0 + h*e) - f(x0 - h*e)) / (2*h);
    end
end
```

```
f = @(x) x(1)^2 + 3*x(1)*x(2) + x(2)^2;
g = numericalGradient(f, [1; 2], 1e-6);
disp(g)
```

MATLAB Lab 2: Visualize a Gradient Vector

```
clear; clc; close all;

f_vec  = @(x) x(1)^2 + 3*x(1)*x(2) + x(2)^2;
f_plot = @(x,y) x.^2 + 3*x.*y + y.^2;

x0 = [1; 2];
g = numericalGradient(f_vec, x0, 1e-6);

[X,Y] = meshgrid(-4:0.1:4, -4:0.1:4);
Z = f_plot(X,Y);

figure;
contour(X,Y,Z,30); hold on;
plot(x0(1), x0(2), 'o', 'MarkerSize', 8, 'LineWidth', 2);
quiver(x0(1), x0(2), g(1), g(2), 0.5, 'LineWidth', 2);
axis equal; grid on;
title('Numerical gradient at one point');
```

Jacobian Matrix: Many Outputs, Many Inputs

For a vector-valued function

$$F : \mathbb{R}^n \to \mathbb{R}^m, \qquad F(\mathbf{x}) = \begin{bmatrix} F_1(\mathbf{x}) \\ F_2(\mathbf{x}) \\ \vdots \\ F_m(\mathbf{x}) \end{bmatrix}.$$

Jacobian shape

$$J(\mathbf{x}) = \begin{bmatrix} \frac{\partial F_1}{\partial x_1} & \frac{\partial F_1}{\partial x_2} & \cdots & \frac{\partial F_1}{\partial x_n} \\ \frac{\partial F_2}{\partial x_1} & \frac{\partial F_2}{\partial x_2} & \cdots & \frac{\partial F_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial F_m}{\partial x_1} & \frac{\partial F_m}{\partial x_2} & \cdots & \frac{\partial F_m}{\partial x_n} \end{bmatrix}.$$

Jacobian Example to Check by Hand

Function

$$F(x, y, z) = \begin{bmatrix} xy + z \\ x^2 + y^2 \\ \sin z + x \end{bmatrix}$$

Jacobian

$$J = \begin{bmatrix} y & x & 1 \\ 2x & 2y & 0 \\ 1 & 0 & \cos z \end{bmatrix}.$$

At $(x, y, z) = (1, 2, 0.5)$

MATLAB should return approximately

$$\begin{bmatrix} 2 & 1 & 1 \\ 2 & 4 & 0 \\ 1 & 0 & 0.8776 \end{bmatrix}.$$

MATLAB Lab 3: Numerical Jacobian Function

Create a file named `numericalJacobian.m`

```
function J = numericalJacobian(F, x0, h)
    y0 = F(x0);
    m = length(y0);    % number of outputs
    n = length(x0);    % number of inputs
    J = zeros(m,n);

    for i = 1:n
        e = zeros(n,1);
        e(i) = 1;
        y_plus  = F(x0 + h*e);
        y_minus = F(x0 - h*e);
        J(:,i) = (y_plus - y_minus) / (2*h);
    end
end
```

MATLAB Lab 4: Use the Jacobian Function

```
clear; clc;

F = @(x) [
    x(1)*x(2) + x(3);
    x(1)^2 + x(2)^2;
    sin(x(3)) + x(1)
];

x0 = [1; 2; 0.5];
J = numericalJacobian(F, x0, 1e-6);

disp('Numerical Jacobian:')
disp(J)
```

Student checkpoint

Which row corresponds to $F_2 = x^2 + y^2$? Which column corresponds to the input z?

Numerical vs. Symbolic Derivatives

Symbolic derivatives

Use when the formula is clean and the symbolic toolbox is available.

- Exact expression.
- Good for checking small examples.
- Can become messy for large systems.

Numerical finite differences

Use when the function is a black-box program or simulation.

- Works even without algebra.
- Common in engineering simulation.
- Step size and computation time matter.

Research habit

Always test numerical derivatives on a small function where you already know the answer.

Speed Lab: Large Matrices

Why Compare MATLAB and Python?

Not a language war

Python can do matrix computation very well, especially with NumPy. MATLAB is still powerful because matrix thinking is built into the language and workflow.

What students should notice:

- Matrix code is much faster than manual loops.
- Vectorized code is usually clearer and shorter.
- Benchmark results depend on hardware and libraries.

Fair comparison rule

Compare MATLAB vectorized code with NumPy vectorized code. Compare loops only to show why matrix programming matters.

A Huge Jacobian We Can Compute Fast

Vector function

Let

$$F(\mathbf{x}) = A\mathbf{x} + \sin(\mathbf{x}), \qquad A \in \mathbb{R}^{n \times n}.$$

Jacobian

$$J(\mathbf{x}) = A + \operatorname{diag}(\cos(\mathbf{x})).$$

Why this is useful for class

It creates a large Jacobian but avoids waiting for thousands of finite-difference function calls. Students see how formula structure and matrix syntax save time.

MATLAB Benchmark Script

```
clear; clc;
rng(1);

n = 2000;
A = randn(n,n);
x = randn(n,1);

f_eval = @() A*x + sin(x);
j_eval = @() A + diag(cos(x));

t_f = timeit(f_eval);
t_j = timeit(j_eval);

fprintf('F(x) time: %.4f seconds\n', t_f);
fprintf('J(x) time: %.4f seconds\n', t_j);
```

Instructor note

If memory is limited, reduce `n` to 1000. A dense 2000×2000 matrix has four million entries.

Python / NumPy Comparison Script

```python
import numpy as np
from timeit import timeit

rng = np.random.default_rng(1)
n = 2000
A = rng.standard_normal((n, n))
x = rng.standard_normal(n)

def f_eval():
    return A @ x + np.sin(x)

def j_eval():
    return A + np.diag(np.cos(x))

t_f = timeit(f_eval, number=10) / 10
t_j = timeit(j_eval, number=10) / 10

print(f"F(x) time: {t_f:.4f} seconds")
print(f"J(x) time: {t_j:.4f} seconds")
```

How to Interpret Benchmark Results

Good conclusions

- Vectorization matters more than language identity.
- Matrix libraries use optimized low-level routines.
- MATLAB encourages students to write matrix-shaped code early.

Bad conclusions

- "One timing proves one language is always faster."
- "Loops are always wrong."
- "Numerical answers are correct because the code ran."

Scientific habit

Benchmark like an experiment: state hardware, matrix size, number of runs, and whether the code is vectorized.

Research Reading Workshop

From Calculus Tools to Research Papers

Why read papers after gradients?

Many AI and smart-grid papers depend on optimization, sensitivity, Jacobians, or gradient-based learning.

- When a paper says "we minimize loss," ask what variables are changing.
- When a paper says "gradient-based," ask what function is being differentiated.
- When a paper says "sensitivity," ask what input perturbation changes which output.

Reading goal

Do not understand every formula today. Learn how to locate the problem, method, and result quickly.

Reading Set for Today

Assigned reading

MIT thesis paper / thesis introduction

- Read the abstract first.
- Then read the introduction paragraph by paragraph.
- Mark the problem sentence, gap sentence, and contribution sentence.

Student-selected reading

Three papers found by the student

- Read each abstract.
- Fill one Problem / Method / Result table row per paper.
- Choose one paper for deeper LLM-assisted reading.

Abstract Breakdown: Problem / Method / Result

Paper	Problem	Method	Result
MIT thesis	What pain point motivates the research?	What tool, model, or experiment is used?	What result is claimed?
Paper 1			
Paper 2			
Paper 3			

Rule

Each cell must be one sentence only. If it takes five sentences, the idea is not clear yet.

How to Use an LLM Without Losing Your Own Brain

Good LLM questions

- "Extract the problem, method, and result from this abstract."
- "List the assumptions made in the introduction."
- "Explain this paragraph for a 9th-grade student."
- "Which claims need experimental evidence?"

Manual validation

- Highlight the exact abstract sentence supporting each answer.
- Check whether the LLM invented missing details.
- Rewrite the final summary in your own words.

In-Class Flow: 120 Minutes

1. **0–25 min:** finish gradient intuition with contour and smart-grid examples.
2. **25–55 min:** MATLAB basics and numerical-gradient live coding.
3. **55–80 min:** numerical Jacobian and large-matrix benchmark discussion.
4. **80–105 min:** MIT thesis abstract and introduction reading.
5. **105–120 min:** three-paper abstract breakdown and LLM validation plan.

Instructor move

Keep switching between "what is the math idea?" and "where would this appear in a real research paper?"

Homework for This Week

1. Create `numericalGradient.m` and `numericalJacobian.m`; test both on the examples from class.
2. Run the MATLAB benchmark with at least two matrix sizes, such as $n = 500$ and $n = 1000$; record timing results.
3. Optional comparison: run the NumPy version and write three sentences interpreting the benchmark fairly.
4. Complete a Problem / Method / Result table for the MIT thesis and the three student-selected papers.
5. Use an LLM to summarize one paper, then validate every major claim against the abstract or introduction.

No-AI coding rule for manual training

When completing the helper functions, write the loop logic yourself before asking an LLM for review.

Thank you!

Next week: signal processing.